

Логирование

ASP NET Core

Ведение лога и ILogger

ASP.NET Core имеет встроенную поддержку логгирования, что позволяет применять логирование с минимальными вкраплениями кода в функционал приложения.

Для логирования данных нам необходим объект **ILogger<T>**. По умолчанию среда ASP.NET Core через механизм внедрения зависимостей уже предоставляет нам такой объект. Его можно получить как и любую другую зависимость в приложении. Также этот объект можно получить через свойство **Logger** объекта **WebApplication**.

Например, используем встроенный логгер для логирования на консоль приложения:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Run(async (context) =>
{
    // пишем на консоль информацию
    app.Logger.LogInformation($"Processing request {context.Request.Path}");

    await context.Response.WriteAsync("Hello World!");
});

app.Run();
```

В данном случае через свойство **app.Logger** получаем встроенный логгер и с помощью его метода **logger.LogInformation** передаем на консоль некоторую информацию.

При обращении к приложению с помощью следующего запроса *http://localhost:xxxxx/hello* на консоль будет выведена информация, переданная логгером:

Категория логгера

При создании логгера для него указывается категория. Обычно в качестве категории логгера выступает класс, в котором используется логгер. В этом случае логгер типизируется классом-категории. Например, логгер, для которого в качестве категории выступает класс Program:

```
ILogger<Program>
```

В чем смысл категории? Категория задает текстовую метку, с которой ассоциируется сообщение логгера, и в выводе лога мы ее можем увидеть.

Где это может быть полезно? Например, у нас есть несколько классов middleware, где ведется логирование. Указывая в качестве категории текущий класс, впоследствии в логе мы можем увидеть, в каком классе именно было создано данное сообщение лога. Поэтому, как правило, в качестве категории указывается текущий класс, но в принципе это необязательно.

Получение логгера через внедрение зависимостей

Поскольку логгер добавляется в сервисы приложения, то мы можем получить его как и любой другой сервис через систему внедрения зависимостей. Например:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/hello", (ILogger<Program> logger) =>
{
    logger.LogInformation($"Path: /hello   Time: {DateTime.Now.ToLongTimeString()}");
    return "Hello World";
});

app.Run();
```

В данном случае при обращении по адресу "/hello" сработает конечная точка, в обработчике которой через механизм внедрения зависимостей можно получить объект логгера. Стоит учитывать, что в этом случае для логгера надо определить категорию. Здесь в качестве категории применяется класс Program (неявный класс, в котором и запускается приложение).

В самом обработчике логгер выводит на консоль путь запроса и время запроса

Уровни и методы логирования

При настройке логирования мы можем установить уровень детализации информации с помощью одного из значений перечисления **LogLevel**. Всего мы можем использовать следующие значения:

- **Trace**: используется для вывода наиболее детализированных сообщений. Подобные сообщения могут нести важную информацию о приложении и его строении, поэтому данный уровень лучше использовать при разработке, но никак не при публикации
- **Debug**: для вывода информации, которая может быть полезной в процессе разработки и отладки приложения
- **Information**: уровень сообщений, позволяющий просто отследить поток выполнения приложения
- **Warning**: используется для вывода сообщений о неожиданных событиях, например, ошибках, которые не останавливают выполнение приложения, но в то же время должны быть исследованы
- **Error**: для вывода информации об ошибках и исключениях, которые возникли при текущей операции и которые не могут быть обработаны
- **Critical**: уровень критических ошибок, которые требуют немедленной реакции - ошибками операционной системы, потерей данных в бд, переполнение памяти диска и т.д.
- **None**: вывод информации в лог не применяется

Уровни и методы логирования

Для вывода соответствующего уровня информации у объекта `ILogger` определены соответствующие методы расширения:

- `LogDebug()`
- `LogTrace()`
- `LogInformation()`
- `LogWarning()`
- `LogError()`
- `LogCritical()`

Так, в примере выше для вывода информации на консоль использовался метод `LogInformation()`.

Вывод сообщений уровня `Trace` по умолчанию отключен.

Каждый такой метод имеет несколько перегрузок, которые могут принимать ряд различных параметров:

- `string data`: строковое сообщение для лога
- `int eventId`: числовой идентификатор, который связан с логом. Идентификатор должен быть статическим и специфическим для определенной части логируемых событий.
- `string format`: строковое сообщения для лога, которое может содержать параметры
- `object[] args`: набор параметров для строкового сообщения
- `Exception error`: логируемый объект исключения

Также для логирования определен общий метод **`Log()`**, который позволяет определить уровень логгера через один из параметров:

```
logger.Log(LogLevel.Information, $"Requested Path: {context.Request.Path}");
```

Уровни и методы логирования

При стандартном логировании на консоль для каждого уровня/метода определен своя метка и цветовой маркер, которые позволяют сразу выделить сообщение соответствующего уровня. Например, при запуске следующего кода:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Run(async (context) =>
{
    var path = context.Request.Path;
    app.Logger.LogCritical($"LogCritical {path}");
    app.Logger.LogError($"LogError {path}");
    app.Logger.LogInformation($"LogInformation {path}");
    app.Logger.LogWarning($"LogWarning {path}");

    await context.Response.WriteAsync("Hello World!");
});

app.Run();
```

Фабрика логгера и провайдеры логирования

В примерах в прошлой теме мы получали объект логгера, который добавляется через DI. Но мы можем также использовать фабрику логгера для его создания:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

ILoggerFactory loggerFactory = LoggerFactory.Create(builder
=> builder.AddConsole());
ILogger logger = loggerFactory.CreateLogger<Program>();
app.Run(async (context) =>
{
    logger.LogInformation($"Requested Path:
{context.Request.Path}");
    await context.Response.WriteAsync("Hello World!");
});

app.Run();
```

В данном случае с помощью метода **LoggerFactory.Create** создается фабрика логгера в виде объекта **ILoggerFactory**. В качестве параметра метод принимает делегат, который устанавливает некоторые настройки логирования. В частности, метод **AddConsole()** объекта **ILoggingBuilder** устанавливает вывод сообщений лога на консоль. Затем метод **CreateLogger()** фабрики собственно создает логгер: **ILogger logger = loggerFactory.CreateLogger<Program>()**; Метод **CreateLogger()** типизируется классом, который представляет категорию. В данном случае это класс **Program**, в котором неявно выполняется данный код. Но в качестве альтернативы название категории можно передать в метод в качестве параметра в виде строки: **ILogger logger = loggerFactory.CreateLogger("WebApplication")**; В итоге мы получим тот же вывод сообщений на консоль. Но преимущество использования фабрики логгеров состоит в том, что мы можем дополнительно настроить различные параметры логирования, в частности, провайдер логирования.

Получение фабрики логгера через DI

Как и логгер, фабрика логгера доступна в приложении в виде сервиса, соответственно ее можно получить через механизм внедрения зависимостей:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Map("/hello", (ILoggerFactory loggerFactory)=>{

    // создаем логгер с категорией "MapLogger"
    ILogger logger = loggerFactory.CreateLogger("MapLogger");
    // логируем некоторое сообщение
    logger.LogInformation($"Path: /hello    Time: {DateTime.Now.ToLongTimeString()}");
    return "Hello World!";
});

app.Run();
```

Провайдеры логирования

В примере выше логирование шло на консоль. Вообще путь логирования определяется провайдером логирования. По умолчанию ASP.NET Core предоставляет следующие провайдеры:

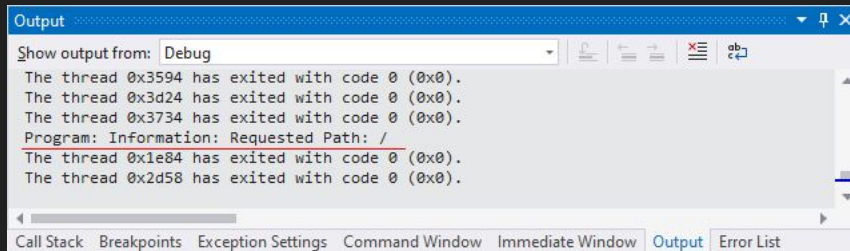
- **Console:** вывод информации на консоль. Устанавливается методом **AddConsole()**
- **Debug:** использует для ведения записей лога класс **System.Diagnostics.Debug** и в частности его метод **Debug.WriteLine**. Соответственно все записи лога мы можем увидеть в окне Output в Visual Studio. Устанавливается методом **AddDebug()**. Стоит отметить, что данный способ работает только при запуске проекта в режиме отладки
- **EventSource:** на Windows введет логирование в лог ETW (Event Tracing for Windows), для просмотра которого может использоваться инструмент PerfView (или аналогичный инструменты). Хотя данный провайдер задумывался как кроссплатформенный, для Linux и MacOS пока назначение лога не определено. Устанавливается методом **AddEventSourceLogger()**
- **EventLog:** записывает в Windows Event Log, соответственно работает только при запуске на Windows. Устанавливается методом **AddEventLog()**

Например, вместо консоли зададим вывод лога в окне Output в Visual Studio:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

var loggerFactory = LoggerFactory.Create(builder =>
    builder.AddDebug());
ILogger logger = loggerFactory.CreateLogger<Program>();
app.Run(async (context) =>
{
    logger.LogInformation($"Requested Path:
{context.Request.Path}");
    await context.Response.WriteAsync("Hello World!");
});

app.Run();
```



Конфигурация и фильтрация логирования

Для логгера можно задать конфигурацию с помощью одного из следующих источников:

- Файлы (json, xml)
- Аргументы командной строки
- Переменные среды окружения
- Объекты .NET
- Незашифрованное хранилище Secret Manager

Также можно создать свой провайдер конфигурации логгера.

Например, по умолчанию в проект по типу Empty добавляется файл **appsettings.json**, который предназначен для конфигурации проекта и в том числе может определять и по умолчанию уже содержит конфигурацию логгера:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Конфигурация и фильтрация логирования

Узел **"Logging"** задает настройки логирования. Далее узел **LogLevel** задает **минимальный уровень** логирования для двух категорий. Категория "Default" представляет универсальное применение ко всем категориям. То есть по умолчанию будут логироваться все сообщения уровня Information. Но для логгера с категорией "Microsoft.AspNetCore" будут логироваться только сообщения уровня Warning.

Мы можем, например, убрать логирование для категории "Microsoft.AspNetCore":

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
    }
  },
  "AllowedHosts": "*"
}
```

Или установить логирование связки различных уровней и категорий для различных провайдеров:

```
{
  "Logging": {
    "Debug": {
      "LogLevel": {
        "Default": "Debug"
      }
    },
    "Console": {
      "LogLevel": {
        "Default": "Information",
        "Microsoft.AspNetCore": "Warning"
      }
    },
    "LogLevel": {
      "Default": "Error"
    }
  },
  "AllowedHosts": "*"
}
```

Конфигурация и фильтрация логирования

Здесь определяются четыре правила. Одно правило определено для провайдера Debug:

```
"Debug": {  
  "LogLevel": {  
    "Default": "Debug"  
  }  
}
```

То есть для провайдера Debug определено логирование сообщения уровня Debug для всех категорий.

Для провайдера Console определено два правила:

```
"Console": {  
  "LogLevel": {  
    "Default": "Information",  
    "Microsoft.AspNetCore": "Warning"  
  }  
}
```

В данном случае для всех категорий будут логироваться сообщения уровня "Information". Для категории "Microsoft.AspNetCore" логируются сообщения уровня "Warning".

И также определено одно общее правило для всех провайдеров:

```
"LogLevel": {  
  "Default": "Error"  
}
```

Конфигурация и фильтрация логирования

Логирование сообщений уровня "Error" для всех категорий.

Следует учитывать, что настройки узла `Logging.{имя_провайдера}.LogLevel` переопределяют настройки в узле `Logging.LogLevel`

Также можно настроить применяемые фильтры программно с помощью фабрики логгера:

```
using Microsoft.Extensions.Logging.Debug;
//.....

var loggerFactory = LoggerFactory.Create(builder =>
{
    builder.AddDebug();
    builder.AddConsole();
    // настройка фильтров
    builder.AddFilter("System", LogLevel.Information)
        .AddFilter<DebugLoggerProvider>("Microsoft",
LogLevel.Trace);
});

ILogger logger = loggerFactory.CreateLogger("WebApplication");
```

Фильтр логирования задается с помощью метода **AddFilter()**, который позволяет задать для определенной категории определенный уровень логирования. Например:

```
builder.AddFilter("System", LogLevel.Information)
```

устанавливает для категории "System" уровень `LogLevel.Information` для всех провайдеров. Это все равно, если бы мы написали в файле `appsettings.json`:

```
"Logging": {
  "LogLevel": {
    "System": "Information"
  }
}
```

Второй вызов `AddFilter` обобщенный - обобщенный класс указывает на провайдер, для которого задается фильтр. То есть для провайдера `DebugLoggerProvider` для категории "Microsoft" устанавливается уровень `LogLevel.Trace`:

```
AddFilter<DebugLoggerProvider>("Microsoft", LogLevel.Trace);
Этот вызов аналогичен следующему определению в файле конфигурации
"Logging": {
  "Debug": {
    "LogLevel": {
      "Microsoft": "Trace"
    }
  }
}
```

Конфигурация и фильтрация логирования

Можно определять фильтры сразу и в коде С#, и в файле конфигурации. При создании логгера объект ILoggerFactory выбирает и применяет одно правило для каждого провайдера. Выбор нужного правила состоит из ряда этапов:

1. Выбираются все правила, которые соответствуют провайдеру. Если таких правил нет, то выбираются все правила, общие для всех провайдеров.
2. Из выбранных на предыдущем шаге правил выбирается правило, которое имеет наиболее длинное соответствие имени категории. Если такого нет, то выбираются все правила, общие для всех категорий.
3. Если на предыдущем шаге выбрано несколько правил, из них выбирается самое последнее (Порядковый номер правила соответствует его порядку определения в файле конфигурации или в коде. Причем правила из файла конфигурации предшествуют правилам, определенным в коде).
4. Если на предыдущем шаге не выбрано никаких правил, то применяется настройка `MinimumLevel`. Данную опцию также можно определить в коде, например, при настройке фабрики логгера:

```
var loggerFactory = LoggerFactory.Create(builder =>
{
    builder.AddDebug();
    builder.AddFilter("System", LogLevel.Debug)
        .SetMinimumLevel(LogLevel.Debug);    // Определение MinimumLevel
});
```

Глобальная настройка логирования

Для глобальной установки настроек логирования у объекта

`WebApplicationBuilder` определено свойство **Logging**, которое представляет тип **ILoggingBuilder**. Для управления логирования он предоставляет ряд методов:

- `AddConfiguration()`: добавляет конфигурацию логгера в виде объекта `IConfiguration`
- `AddConsole()`: добавляет консольный логгер
- `AddConsoleFormatter()`: добавляет объект форматирования для консольного вывода
- `AddJsonConsole()`: добавляет форматирование сообщений консольного логгера в формат "json"
- `AddSimpleConsole()`: добавляет простое форматирование для логгирования на консоль
- `AddDebug()`: добавляет логгер отладки
- `AddEventLog()`: добавляет логгер для вывод в журнал Windows Event Log
- `AddEventSourceLogger()`: добавляет логгер для вывода в лог Event Tracing for Windows
- `AddFilter()`: добавляет фильтрацию для сообщений логгера
- `AddProvider()`: добавляет провайдер логирования
- `ClearProviders()`: удаляет все зарегистрированные провайдеры логирования
- `SetMinimumLevel()`: устанавливает минимальный уровень логирования

```
var builder = WebApplication.CreateBuilder();
builder.Logging.ClearProviders(); // удаляем все провайдеры
builder.Logging.AddConsole();   // добавляем провайдер для
// логгирования на консоль
var app = builder.Build();
```

Создание провайдера логирования

Стандартная инфраструктура ASP NET Core предоставляет, возможно, не самые удобные способы логирования - на консоль, в окне Output в Visual Studio. Однако в то же время ASP.NET Core позволяет полностью определить свою логику ведения лога. Допустим, мы хотим сохранять сообщения в текстовом файле.

Вначале добавим в проект новый класс **FileLogger**:

```
public class FileLogger : ILogger, IDisposable
{
    string filePath;
    static object _lock = new object();
    public FileLogger(string path)
    {
        filePath = path;
    }
    public IDisposable BeginScope<TState>(TState state)
    {
        return this;
    }

    public void Dispose() { }

    public bool IsEnabled(LogLevel logLevel)
    {
        //return logLevel == LogLevel.Trace;
        return true;
    }

    public void Log<TState>(LogLevel logLevel, EventId eventId,
        TState state, Exception? exception, Func<TState, Exception?, string> formatter)
    {
        lock (_lock)
        {
            File.AppendAllText(filePath, formatter(state, exception) + Environment.NewLine);
        }
    }
}
```

Создание провайдера логирования

Класс логгера должен реализовать интерфейс **ILogger**. Этот интерфейс определяет три метода:

- **BeginScope**: этот метод возвращает объект **IDisposable**, который представляет некоторую область видимости для логгера. В данном случае нам этот метод не важен, поэтому возвращаем значение **this** - ссылку на текущий объект класса, который реализует интерфейс **IDisposable**.
- **IsEnabled**: возвращает значения **true** или **false**, которые указывают, доступен ли логгер для использования. Здесь можно задать различную логику. В частности, в этот метод передается объект **LogLevel**, и мы можем, к примеру, задействовать логгер в зависимости от значения этого объекта. Но в данном случае просто возвращаем **true**, то есть логгер доступен всегда.
- **Log**: этот метод предназначен для выполнения логирования. Он принимает пять параметров:
 - **LogLevel**: уровень детализации текущего сообщения
 - **EventId**: идентификатор события
 - **TState**: некоторый объект состояния, который хранит сообщение
 - **Exception**: информация об исключении
 - **formatter**: функция форматирования, которая с помощью двух предыдущих параметров позволяет получить собственно сообщение для логирования
- И в данном методе как раз и производится запись в текстовый файл. Путь к этому файлу передается через конструктор

Далее добавим в проект класс **FileLoggerProvider**:

```
public class FileLoggerProvider : ILoggerProvider
{
    string path;

    public FileLoggerProvider(string path)
    {
        this.path = path;
    }

    public ILogger CreateLogger(string categoryName)
    {
        return new FileLogger(path);
    }

    public void Dispose() {}
}
```

Этот класс предоставляет провайдер логирования. Он должен реализовать интерфейс **ILoggerProvider**. В этом интерфейсе определены два метода:

- **CreateLogger**: создает и возвращает объект логгера. Для создания логгера используется путь к файлу, который передается через конструктор
- **Dispose**: управляет освобождением ресурсов. В данном случае пустая реализация

Создание провайдера логирования

Теперь создадим вспомогательный класс **FileLoggerExtensions**:

```
public static class FileLoggerExtensions
{
    public static ILoggingBuilder AddFile(this
        ILoggingBuilder builder, string filePath)
    {
        builder.AddProvider(new
            FileLoggerProvider(filePath));
        return builder;
    }
}
```

Этот класс добавляет к объекту `ILoggingBuilder` метод расширения `AddFile`, который будет добавлять наш провайдер логирования.

Теперь используем провайдер в файле **Program.cs**:

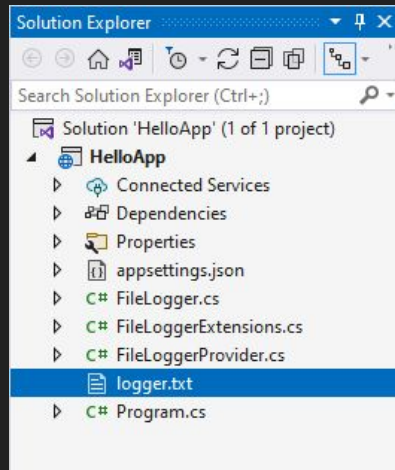
За глобальную установку настроек логирования отвечает свойство **Logging** класса `WebApplicationBuilder`. Это свойство представляет объект `ILoggingBuilder` и предоставляет ряд методов для управления логированием. И в данном случае с помощью выше определенного метода `AddFile` добавляем логирование в файл.

Стоит отметить, что глобальная настройка логирования должна идти до создания объекта `WebApplication`.

```
var builder = WebApplicationBuilder.CreateBuilder();
// устанавливаем файл для логирования
builder.Logging.AddFile(Path.Combine(Directory.GetCurrentDir
    ectory(), "logger.txt"));
// настройка логирования с помощью свойства Logging идет до
// создания объекта WebApplication
var app = builder.Build();

app.Run(async (context) =>
{
    app.Logger.LogInformation($"Path: {context.Request.Path}
        Time:{DateTime.Now.ToLongTimeString()}");
    await context.Response.WriteAsync("Hello World!");
});

app.Run();
```



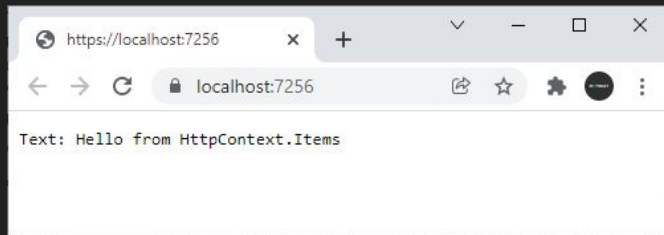
Состояние приложения. Куки. Сессии. HttpContext.Items

Приложение ASP.NET Core может хранить некоторое состояние. Это могут быть как какие-то глобальные данные, так и данные, которые непосредственно относятся к запросу и пользователю. И в зависимости от вида данных, существуют различные способы для их хранения.

Один из простейших способов хранения данных представляет коллекция **HttpContext.Items** - объект типа `IDictionary<object, object>`. Эта коллекция предназначена для таких данных, которые непосредственно связаны с текущим запросом. После завершения запроса все данные из `HttpContext.Items` удаляются. Каждый объект в этой коллекции имеет ключ и значение. И с помощью ключей можно управлять объектами коллекции.

В каком случае мы можем применить данную коллекцию? Например, если у нас обработка запроса вовлекает множество компонентов `middleware`, и мы хотим, чтобы для этих компонентов были доступны общие данные, то как раз можем применить эту коллекцию. Например, пусть в приложении определен следующий код:

```
var builder =  
    WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.Use(async (context, next) =>  
{  
    context.Items["text"] = "Hello from  
    HttpContext.Items";  
    await next.Invoke();  
});  
  
app.Run(async (context) => await  
context.Response.WriteAsync($"Text:  
{context.Items["text"]}"));  
  
app.Run();
```



HttpContext.Items

Здесь в одном middleware определяется ключ "text" со значением "Hello from HttpContext.Items":

```
context.Items["text"] = "Hello from HttpContext.Items";
```

В другом middleware этот объект используется для установки отправляемого ответа.

HttpContext.Items предоставляет ряд методов для управления элементами:

- void Add(object key, object value): добавляет объект value с ключом key
- void Clear(): удаляет все объекты
- bool ContainsKey(object key): возвращает true, если словарь содержит объект с ключом key
- bool Remove(object key): удаляет объект с ключом key, в случае удачного удаления возвращает true
- bool TryGetValue(object key, out object value): возвращает true, если значение объекта с ключом key успешно получено в объект value

Применение некоторых методов:

```
var builder = WebApplication.CreateBuilder();
var app = builder.Build();

app.Use(async (context, next) =>
{
    context.Items.Add("message", "Hello World");
    await next.Invoke();
});

app.Run(async (context) =>
{
    if (context.Items.ContainsKey("message"))
        await context.Response.WriteAsync($"Message: {context.Items["message"]}");
    else
        await context.Response.WriteAsync("Random Text");
});

app.Run();
```

Cookie (Куки)

Куки представляют самый простой способ сохранить данные пользователя. Куки хранятся на компьютере пользователя и могут устанавливаться как на сервере, так и на клиенте. Так как куки посылаются с каждым запросом на сервер, то их максимальный размер ограничен 4096 байтами.

Для работы с куками также можно использовать контекст запроса `HttpContext`, который передается в качестве параметра в компоненты `middleware`, а также доступен в контроллерах и `Razor Pages`.

Чтобы получить куки, которые приходят вместе с запросом к приложению, нам надо использовать коллекцию **`Request.Cookies`** объекта `HttpContext`. Эта коллекция представляет объект **`IRequestCookieCollection`**, в котором каждый элемент - это объект **`KeyValuePair<string, string>`**, то есть некоторую пару ключ-значение.

Для этой коллекции определено несколько методов:

- `bool ContainsKey(string key)`: возвращает `true`, если в коллекции кук есть куки с ключом `key`
- `bool TryGetValue(string key, out string value)`: возвращает `true`, если удалось получить значение куки с ключом `key` в переменную `value`

Стоит отметить, что куки - это строковые значения. Неважно, что вы пытаетесь сохранить в куки - все это необходимо приводить к строке и соответственно получаете из кук вы тоже строки.

Например, получим куку "name":

```
if (context.Request.Cookies.ContainsKey("name"))  
    string name = context.Request.Cookies["name"];
```

Cookie (Куки)

Для установки кук, которые отправляются в ответ клиенту, применяется объект `context.Response.Cookies`, который представляет интерфейс `IResponseCookies`. Этот интерфейс определяет два метода:

- **Append(string key, string value)**: добавляет для куки с ключом `key` значение `value`
- **Delete(string key)**: удаляет куку по ключу

Например, объединим в приложении установку и получение кук

При получении запроса мы смотрим, установлена ли кука "name":

```
if (context.Request.Cookies.ContainsKey("name"))
```

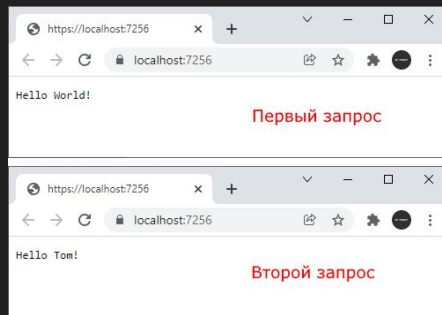
Если кука не установлена (например, при первом обращении к приложению), то устанавливаем ее и отправляем пользователю в ответ строку "Hello World!".

```
context.Response.Cookies.Append("name", "Tom");  
await context.Response.WriteAsync("Hello World!");
```

Если кука установлена, то получаем ее значение и отправляем его пользователю

```
string? name = context.Request.Cookies["name"];  
await context.Response.WriteAsync($"Hello {name}!");
```

```
var builder = WebApplication.CreateBuilder();  
var app = builder.Build();  
  
app.Run(async (context) =>  
{  
    if (context.Request.Cookies.ContainsKey("name"))  
    {  
        string? name = context.Request.Cookies["name"];  
        await context.Response.WriteAsync($"Hello {name}!");  
    }  
    else  
    {  
        context.Response.Cookies.Append("name", "Tom");  
        await context.Response.WriteAsync("Hello World!");  
    }  
});  
  
app.Run();
```



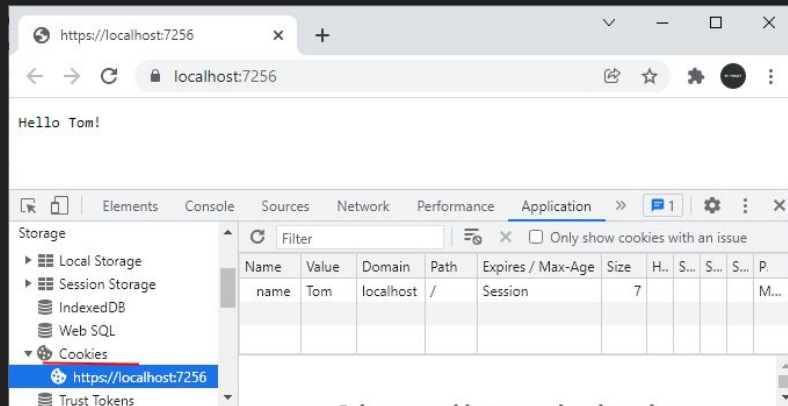
Cookie (Куки)

После установки кук в результате первого запроса в браузере будет установлена кука `name`. И через средства разработчика в веб-браузере мы сможем увидеть ее значение (если браузер поддерживает подобное). Например, в Google Chrome посмотреть куки можно на вкладке **Application** -> **Cookies**:

В примере выше мы также могли бы применить метод `TryGetValue()` для получения кук:

```
app.Run(async (context) =>
{
    if (context.Request.Cookies.TryGetValue("name", out var login))
    {
        await context.Response.WriteAsync($"Hello {login}!");
    }
    else
    {
        context.Response.Cookies.Append("name", "Tom");
        await context.Response.WriteAsync("Hello World!");
    }
});
```

Стоит отметить, что пользователь через подобные инструменты (если веб-браузер позволяет) может вручную поменять значение кук, либо вовсе удалить их.



Сессия

Сессия представляет собой ряд последовательных запросов, совершенных в одном браузере в течение некоторого времени. Сессия может использоваться для сохранения каких-то временных данных, которые должны быть доступны, пока пользователь работает с приложением, и не требуют постоянного хранения.

Для хранения состояния сессии на сервере создается словарь или хеш-таблица, которая хранится в кэше и которая существует для всех запросов из одного браузера в течение некоторого времени. На клиенте хранится идентификатор сессии в куках. Этот идентификатор посылается на сервер с каждым запросом. Сервер использует этот идентификатор для извлечения нужных данных из сессии. Эти куки удаляются только при завершении сессии. Но если сервер получает куки, которые установлены уже для истекшей сессии, то для этих кук создается новая сессия.

Сервер хранит данные сессии в течение ограниченного промежутка времени после последнего запроса. По умолчанию этот промежуток равен 20 минутам, хотя его также можно изменить.

Следует учитывать, что данные сессии специфичны для одного браузера и не разделяются между браузерами. То есть для каждого браузера на одном компьютере будет создаваться свой набор данных.

Все сессии работают поверх объекта **IDistributedCache**, и ASP.NET Core предоставляет встроенную реализацию IDistributedCache, которую мы можем использовать. Для этого определим в приложении следующий код

Сессия

```
var builder = WebApplication.CreateBuilder();

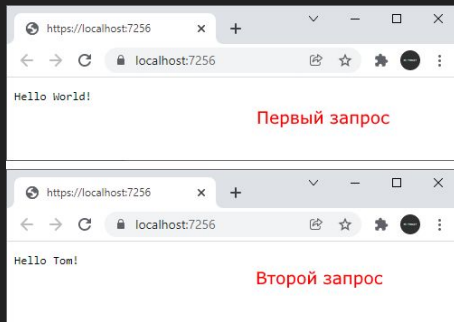
builder.Services.AddDistributedMemoryCache(); // добавляем
IDistributedMemoryCache
builder.Services.AddSession(); // добавляем сервисы сессии

var app = builder.Build();

app.UseSession(); // добавляем middleware для работы с сессиями

app.Run(async (context) =>
{
    if (context.Session.Keys.Contains("name"))
        await context.Response.WriteAsync($"Hello
(context.Session.GetString("name"))!");
    else
    {
        context.Session.SetString("name", "Tom");
        await context.Response.WriteAsync("Hello World!");
    }
});

app.Run();
```



Сначала добавляем необходимые сервисы:

```
builder.Services.AddDistributedMemoryCache(); // добавляем
IDistributedMemoryCache
builder.Services.AddSession(); // добавляем сервисы сессии
```

Затем с помощью метода **UseSession()** встраиваем в конвейер обработки запроса middleware для работы с сессиями:

```
app.UseSession(); // добавляем middleware для работы с сессиями
```

```
app.Run(async (context) =>
{
    if (context.Session.Keys.Contains("name"))
        await context.Response.WriteAsync($"Hello
(context.Session.GetString("name"))!");
    else
    {
        context.Session.SetString("name", "Tom");
        await context.Response.WriteAsync("Hello World!");
    }
});
```

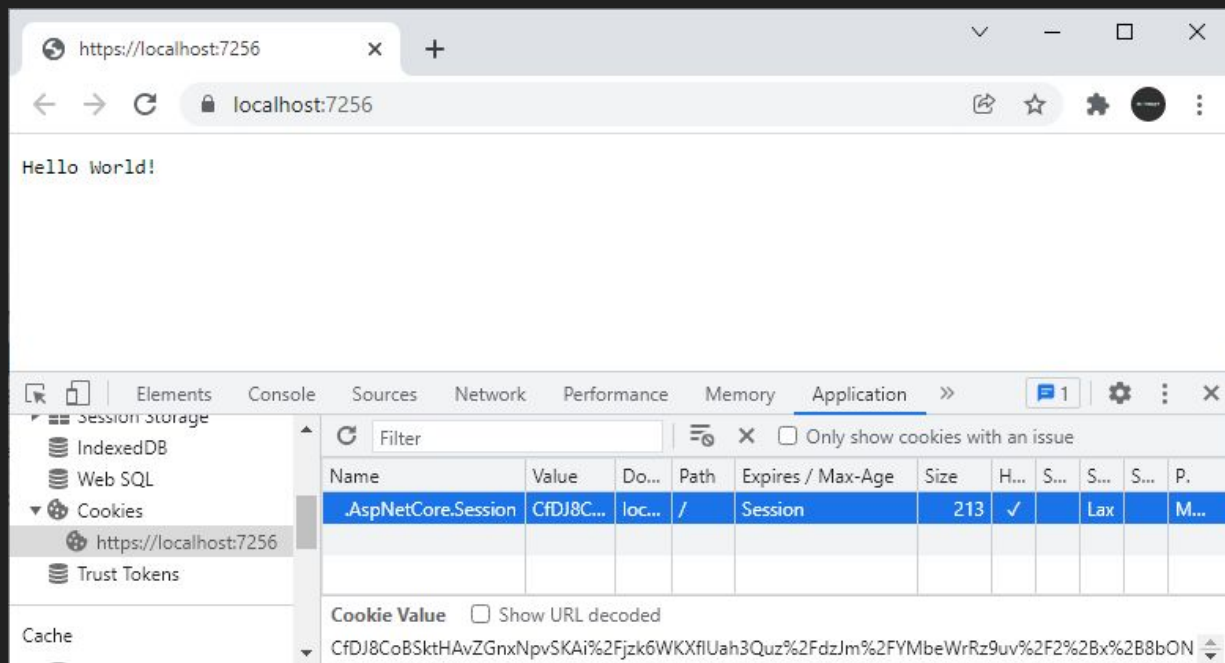
Если мы вдруг не используем `app.UseSession()` или попробуем обратиться к сессии до применения этого метода, то получим исключение `InvalidOperationException`.

После вызова `app.UseSession()` мы сможем управлять сессиями через свойство

HttpContext.Session, которое представляет объект интерфейса **ISession**. В данном случае мы проверяем, определен ли в сессии ключ "name". Если ключ определен, то передаем ответ значение по этому ключу. Если ключ не определен, устанавливаем его.

Сессия

После первого запроса мы сможем через инструменты браузера для разработчиков найти куку **AspNetCore.Session**, которая будет представлять идентификатор сессии:



Управление сессией

Объект **ISession** определяет ряд свойств и методов, которые мы можем использовать:

- **Keys**: свойство, представляющее список строк, который хранит все доступные ключи
- **Clear()**: очищает сессию
- **Get(string key)**: получает по ключу key значение, которое представляет массив байтов
- **GetInt32(string key)**: получает по ключу key значение, которое представляет целочисленное значение
- **GetString(string key)**: получает по ключу key значение, которое представляет строку
- **Set(string key, byte[] value)**: устанавливает по ключу key значение, которое представляет массив байтов
- **SetInt32(string key, int value)**: устанавливает по ключу key значение, которое представляет целочисленное значение value
- **SetString(string key, string value)**: устанавливает по ключу key значение, которое представляет строку value
- **Remove(string key)**: удаляет значение по ключу

Настройка сессии

Для разграничения сессий для них устанавливается идентификатор. Каждая сессия имеет свой идентификатор, который сохраняется в куках. По умолчанию эти куки имеют название "AspNet.Session". И также по умолчанию куки имеют настройку `CookieHttpOnly=true`, поэтому они не доступны для клиентских скриптов из браузера. Но мы можем переопределить ряд настроек сессии с помощью свойств объекта **SessionOptions**:

- **Cookie.Name**: имя куки
- **Cookie.Domain**: домен, для которого устанавливаются куки
- **Cookie.HttpOnly**: доступны ли куки только при передаче через HTTP-запрос
- **Cookie.Path**: путь, который используется куками
- **Cookie.Expiration**: время действия куки в виде объекта `System.TimeSpan`
- **Cookie.IsEssential**: при значении `true` указывает, что куки критичны и необходимы для работы этого приложения
- **IdleTimeout**: время действия сессии в виде объекта `System.TimeSpan` при неактивности пользователя. При каждом новом запросе таймаут сбрасывается. Этот параметр не зависит от `Cookie.Expiration`.

Для использования этих свойств изменим установку сервисов:

```
builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession(options =>
{
    options.Cookie.Name = ".MyApp.Session";
    options.IdleTimeout = TimeSpan.FromSeconds(3600);
    options.Cookie.IsEssential = true;
});
```

Хранение сложных объектов

В случае выше в сессиях хранились простые строки. Если же надо сохранить какой-то сложный объект, то его надо сериализовать в строку, а при получении из сессии - обратно десериализовать. Как правило, для этого определяются методы расширения для объекта `ISession`. В частности, добавим следующий класс:

```
using System.Text.Json;

public static class SessionExtensions
{
    public static void Set<T>(this ISession session, string key, T value)
    {
        session.SetString(key, JsonSerializer.Serialize<T>(value));
    }

    public static T? Get<T>(this ISession session, string key)
    {
        var value = session.GetString(key);
        return value == null ? default(T) :
        JsonSerializer.Deserialize<T>(value);
    }
}
```

Метод `Set` сохраняет в сессию данные, а метод `Get` извлекает их из сессии.

Допустим, у нас есть класс `Person`:

```
class Person
{
    public string Name { get; set; } = "";
    public int Age { get; set; }
}
```

Хранение сложных объектов

В приложении выполним сериализацию и десериализацию объекта в сессию:

```
var builder = WebApplication.CreateBuilder();

builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession();

var app = builder.Build();

app.UseSession();

app.Run(async (context) =>
{
    if (context.Session.Keys.Contains("person"))
    {
        Person? person = context.Session.Get<Person>("person");
        await context.Response.WriteAsync($"Hello {person?.Name}, your age: {person?.Age}!");
    }
    else
    {
        Person person = new Person { Name = "Tom", Age = 22 };
        context.Session.Set<Person>("person", person);
        await context.Response.WriteAsync("Hello World!");
    }
});

app.Run();
```

